

Cours arbres

Contents

I)	Arbres binaires	1
I.1)	Définitions	1
I.2)	Propriétés des arbres	2
I.3)	Et en OCaml ?	3
I.4)	Preuves sur les arbres binaires	4
II)	Arbres généraux	4
II.1)	Définition	4
II.2)	Implémentation	5
III)	Les parcours d'arbre	5
III.1)	Parcours en profondeur	6
III.2)	Parcours en largeur	6
III.3)	Utilité	6
IV)	Implémentation d'un dictionnaire avec un arbre binaire de recherche	6
IV.1)	Définition	6
IV.2)	Implémentation OCaml	7
IV.3)	Complexité des opérations	7

I) Arbres binaires

I.1) Définitions

Définition 1 [Arbre binaire]

Un arbre binaire stockant des données d'un ensemble E est défini par induction comme :

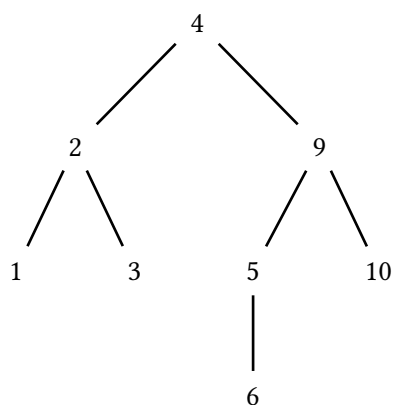
- Soit l'arbre vide
- Soit une feuille contenant un élément de E .
- Soit un noeud interne contenant un ou deux arbres et un élément de E .

On appelle *noeud* soit un noeud interne, soit une feuille.

Les éléments de E sont appelés *étiquette* d'un noeud, et les deux arbres associé à un noeud interne sont communément appelés *fil droit* et *fil gauche*.

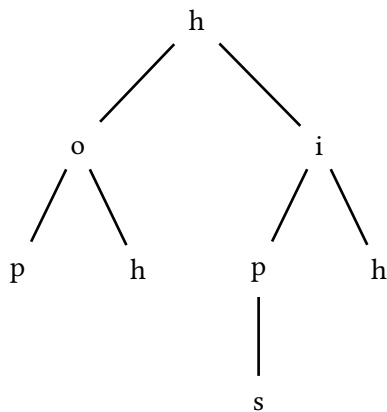
Exemple 2

Arbre binaire de recherche stockant les nombres 1, 2, 3, 4, 5, 6, 9, 10.



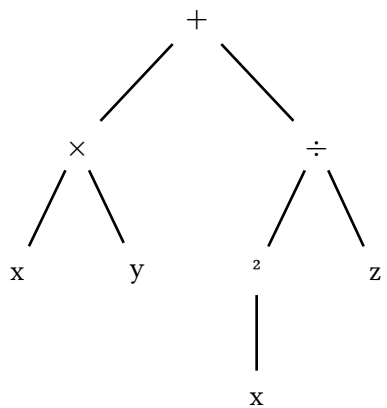
Exemple 3

Arbre préfixe représentant les onomatopées “hop” “hoh” “hip” “hips” et “hih”.



Exemple 4

Arbre d'opération de $x \times y + \frac{x^2}{z}$.



Pour ces trois exemples, la structure de l'arbre est identique, mais les données stockées sont de nature différente

Définition 5 [Arbre binaire strict]

Un arbre binaire *strict* est un arbre binaire où tous les noeuds internes ont exactement deux fils. Attention aux définitions, parfois on appelle “arbre binaire” des arbres binaires strictes.

I.2) Propriétés des arbres

Définition 6 [racine]

La *racine* d'un arbre est son noeud d'origine (le plus en haut dans la représentation).

Exemple 7

On va donner pour exemple l'arbre de l'ex 2.

4 est sa racine

Définition 8 [profondeur]

La *profondeur* d'un noeud est sa distance à la racine.

Exemple 9

- 4 est à une profondeur de 0.
- 5 est à une profondeur de 2.

Définition 10 [taille]

La *taille* d'un arbre est son nombre de noeuds.

Exemple 11

Ici, 8.

Définition 12 [hauteur]

La hauteur d'un arbre est la profondeur maximum des noeuds de l'arbre.

Par définition, l'arbre vide a une hauteur de -1 .

Exemple 13

Ici, 3, car 6 est à une profondeur 3.

Définition 14 [Père, fils, descendant, sous-arbre enraciné]

- Un noeud est le fils d'un autre si c'est son fils droit ou son fils gauche. Ce dernier est alors le père de son fils.
- Les descendants d'un noeuds sont ses fils, et les descendants de ses fils.
- Le sous arbre enraciné en un noeud est composé de ce noeud et de tous ses descendants.

I.3) Et en OCaml ?

On peut définir des arbres binaires de la manière suivante :

```
type arbre_binaire =  
  Vide  
  | N of arbre_binaire * int * arbre_binaire
```

Un noeud avec un seul fils peut être représenté par un fils vide. Une feuille peut être représenté par deux fils vides.

Les arbres binaires stricts non vides sont le plus souvent représentés par :

```
type arbre_binaire_strict =  
  | F of int  
  | N of arbre_binaire_strict * int * arbre_binaire_strict
```

C'est la forme la plus courante d'arbre binaires.

Les fonctions sur les propriétés les plus courantes peuvent être définies de la manière suivante :

```
let rec taille a = match a with  
  | F _ -> 1  
  | N (fg, _, fd) -> (taille fg) + (taille fd)  
;;  
  
let rec hauteur a = match a with  
  | F _ -> 0  
  | N (fg, _, fd) -> max (taille fg) (taille fd)  
;;
```

I.4) Preuves sur les arbres binaires

Exemple 15

Soit A un arbre binaire. On note $T(A)$ sa taille et $h(A)$ sa hauteur.

Montrer que $T(A) \leq 2^{h(A)+1} - 1$

On va faire une preuve par *récurrence forte*:

- (cas de base) si F est une feuille $T(F) = 1 \leq 2^{h(F)+1} - 1 = 1$
- (hérédité) soit $A = (G, D)$ un arbre de hauteur n . Comme G et D sont de hauteur au plus $n - 1$, on a par récurrence : $T(G) \leq 2^{h(G)+1} - 1$ et $T(D) \leq 2^{h(D)+1} - 1$

Donc

$$\begin{aligned} T(A) &= T(G) + T(D) + 1 \\ &\leq 2^{h(G)+1} - 1 + 2^{h(D)+1} - 1 + 1 \\ &\leq 2^{\max(h(G), h(D))+2} - 1 \\ &\leq 2^{h(A)+1} - 1 \end{aligned}$$

Cela conclut la preuve.

Exemple 16

Soit A un arbre binaire. On note $F(A)$ son nombre de feuilles et $h(A)$ sa hauteur.

Montrer que $F(A) \leq 2^{h(A)}$.

- (initialisation) Si A est une feuille, $F(A) = 1 \leq 2^0$
- (hérédité) soit $A = (G, D)$ un arbre de hauteur n . Comme G et D sont de hauteur au plus $n - 1$, on a par récurrence : $F(G) \leq 2^{h(G)}$ et $F(D) \leq 2^{h(D)}$.

Donc

$$\begin{aligned} F(A) &= F(G) + F(D) \\ &\leq 2^{h(G)} + 2^{h(D)} \\ &\leq 2^{h(A)-1} + 2^{h(A)-1} \\ &\leq 2^{h(A)} \end{aligned}$$

Cela conclut la preuve.

II) Arbres généraux

II.1) Définition

Définition 17 [Arbre général]

Un arbre général, ou simplement "arbre", stockant des données d'un ensemble E , est défini par induction comme :

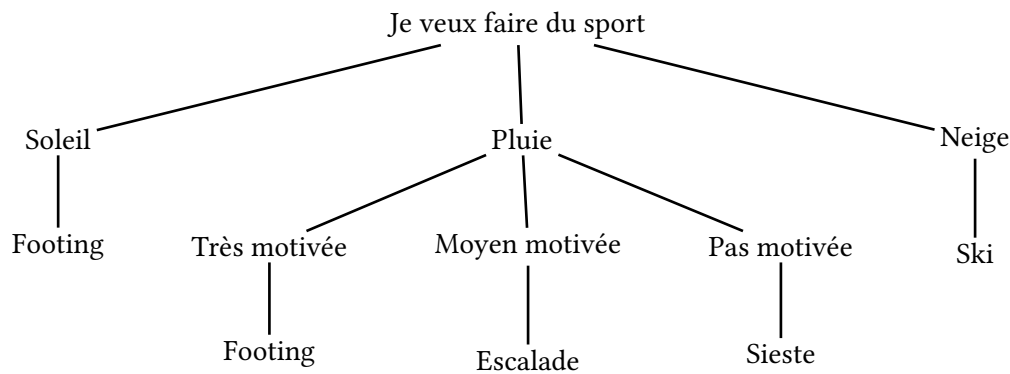
- Soit l'arbre vide
- Soit un noeud contenant un élément de E , ainsi qu'un nombre quelconques d'arbres non vides (ses fils).

Un noeud avec aucun fils est une feuille. Un noeud avec au moins un fils est un noeud interne.

Le vocabulaire sur les arbres est identique à celui des arbres binaires.

Exemple 18

Arbre de décision



II.2) Implémentation

On utilise le type suivant :

```
type arbre =
  | Vide
  | N of int * arbre list
```

Cela complique un petit peu toutes les fonctions ! La dernière fois, on traitait explicitement chacun des fils de notre arbre. Ici, ce n'est pas possible !

Deux solutions :

- une fonction auxiliaire
- être astucieux sur la récurrence

Exemple 19

Calculer la somme des noeuds d'un arbre général.

```
let rec somme (arb: arbre): int =
  (*calcule la somme des sommes des arbres dans une liste d'arbres*)
  let rec aux (l: arbre list) -> int =
    | [] -> 0
    | a::q -> (somme a) + (aux q)
  in
  match arb with
  | Vide -> 0
  | N (x, l) -> x + (somme l)
;;

let rec somme (arb: arbre): int =
  match arb with
  | Vide -> 0
  | N (x, []) -> x
  | N (x, a::q) -> (somme a) + (somme (N (x, q)))
;;
```

III) Les parcours d'arbre

Un parcours d'arbre, c'est un ordre dans lequel on va visiter les noeuds de l'arbre. Il en existe deux principaux.

Cette partie est valable autant pour les arbres binaires que les arbres généraux. On traitera uniquement les concepts généraux des deux parcours principaux dans ce cours; ils seront revu plus en détails lorsqu'on parlera de graphes.

voir <https://visualgo.net/en/dfsbfbs>.

III.1) Parcours en profondeur

Le parcours en profondeur est celui que vous avez utilisé avec des fonctions récursives, par exemple la fonction `taille` d'un arbre binaire. Il s'agit d'aller le plus loin possible, et de ne faire demi-tour que lorsqu'on arrive à une feuille.

III.2) Parcours en largeur

Au contraire, le parcours en largeur consiste à parcourir d'abord tous les noeuds de profondeur 0, puis de profondeur 1, 2, etc.

Il est très compliqué à implémenter de manière récursive, et de même de manière impérative ce n'est pas simple.

III.3) Utilité

Globalement, le parcours en profondeur a l'avantage principal d'être plus simple à implémenter, et c'est donc celui qu'on utilise quand l'ordre des sommets parcouru n'a pas d'importance.

Le parcours en largeur a des avantages en revanche, en particulier sur des arbres très grands : si notre arbre a une hauteur extrêmement grande, mais qu'on ne veut parcourir que les sommets proches de la racine, c'est lui qu'on utilisera.

IV) Implémentation d'un dictionnaire avec un arbre binaire de recherche

IV.1) Définition

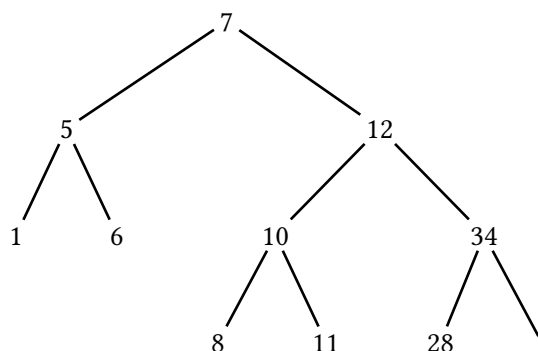
Définition 20 [Arbre binaire de recherche]

Un arbre binaire de recherche est un arbre binaire étiqueté par un ensemble ordonné E , qui, si e est l'étiquette d'un noeud s :

- pour tout noeud descendant du fils gauche de s d'étiquette e_g , $e_g < e$
- pour tout noeud descendant du fils droit de s d'étiquette e_d , $e_d > e$

Exemple 21

Arbre binaire de recherche contenant les entiers 1, 5, 6, 7, 8, 10, 11, 12, 28, 34



IV.2) Implémentation OCaml

(*le type ocaml d'un arbre binaire de recherche sur les entiers*)

```
type abr =  
  | Vide  
  | N of abr * int * abr;;
```

(*recherche d'une valeur dans un arbre binaire de recherche*)

```
let rec recherche (a: abr) (x: int): bool = match a with  
  | Vide -> false  
  | N (ag, y, ad) -> if x = y then true  
    else if x < y then recherche ag x  
    else recherche ad x;;
```

(*ajout d'une valeur dans un abr*)

```
let rec ajout (a: abr) (x: int): abr = match a with  
  | Vide -> N (Vide, x, Vide)  
  | N (ag, y, ad) -> if x = y then a  
    else if x < y then N (ajout ag x, y, ad)  
    else N (ag, y, ajout ad x);;
```

On peut également construire un arbre binaire de recherche à partir d'une liste ne réalisant des ajouts successifs à partir de l'arbre vide.

IV.3) Complexité des opérations

Les opérations d'ajout et de suppression s'appellent récursivement une fois pour chaque niveau de l'arbre. Si on note h la hauteur de l'arbre, elles sont donc en $O(h)$.

Cas 1 : l'arbre est complet

Supposons qu'on ait un arbre A , tel que toutes les feuilles soient à la même profondeur. On dit qu'il est complet. Exprimer la hauteur h de l'arbre en fonction de n sa taille.

$$h = \log_2(n + 1) - 1$$

Preuve par récurrence

- ($h = 0$) alors, $n = 1$, et la relation est vérifiée
- On suppose la propriété vraie pour les arbres complets de hauteur plus petite que $h > 0$. Soit A un arbre de hauteur h .

Soit A_g et A_d ses sous arbres. Ils sont complets, et de hauteur $h - 1$

On a donc :

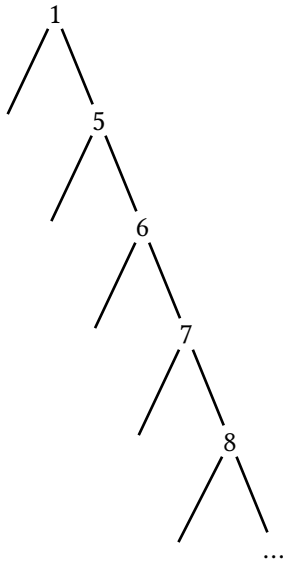
$$\begin{aligned} n &= 1 + n_g + n_d \\ \log_2(n + 1) &= \log_2(n_g + n_d + 2) && \text{or, } n_g = n_d \\ &= \log_2(2 * (n_g + 1)) \\ &= \log_2(n_g + 1) + 1 \\ &= h - 1 + 1 + 1 && \text{(on applique la récurrence)} \\ \log_2(n + 1) - 1 &= h \end{aligned}$$

Ce qui conclut la récurrence.

Dans ce cas, les opérations sur l'arbre binaire de recherche se font en $O(\log(n))$ où n est la taille de l'arbre. C'est efficace !

Cas 2 : l'arbre est une branche

Reprenons l'exemple 21, mais en insérant simplement les entiers dans l'ordre croissant.



Dans ce cas, la hauteur de l'arbre est égale à sa taille. On a des opérations en temps linéaire, pas mieux qu'une simple liste !

Cas 3 : et en général ?

En pratique, avec des éléments aléatoires, on va souvent se retrouver dans un cas proche du 1. Mais pour le garantir, il existe des algorithmes avancés, qui permettent de toujours avoir un arbre équilibré, et d'avoir des complexité en $O(\log(n))$ quoi qu'il arrive.